1.

Pseudocode

```
ALGORITHM  UniqueRandom(n)
//Generates n distinct random integers from 1..4n
//Input: A positive integer n
//Output: An array A[0..n-1] of n distinct integers in [1, 4n]
filled ← 0
while filled < n do
   c ← Random(1, 4n)
    dup ← false
    j ← 0
   while j < filled and dup = false do
     if A[j] = c then dup ← true
     j ← j + 1
    if dup = false then
   A[filled] ← c
     filled ← filled + 1
return A
```

Description

The algorithm makes an array A to hold the numbers, and a counter called filled, it keeps track of how many numbers are saved so far. It picks a random number from 1 to 4n. Then it checks if that number is already in A. If it is then the number is thrown away, and a new one is picked. If it isn't, then the number is saved in A. This repeats until A has n amount of numbers. Check for repeats is done by looking through the array at one number at a time.

2.

Code

```
import java.util.Random;

public class Milestone_one {

private final int n; // how many numbers we want
private final int range; // biggest number allowed (4 times n)
private final Random rng; // the random number maker

// counts how many comparisons we do while checking for repeats
private long comparisons = 0;

// use this one for a normal random list
public Milestone_one(int n) {
this(n, new Random());
}

// use this one if you want the same list every time
public Milestone_one(int n, long seed) {
this(n, new Random(seed));
}
```

```
private Milestone_one(int n, Random rng) {
if (n < 1) {
throw new IllegalArgumentException("n must be at least 1");
}
this.n = n;
this.range = 4 * n;
this.rng = rng;
}

// makes the array of n different random numbers
public int[] generate() {
int[] a = new int[n];
comparisons = 0;
int filled = 0; // how many numbers are saved so far

while (filled < n) {
// pick a random number from 1 to 4n
int candidate = rng.nextInt(range) + 1;

// look through the saved numbers to see if we already have it
boolean duplicate = false;
for (int j = 0; j < filled; j++) {
comparisons++;
if (a[j] == candidate) {
duplicate = true;
break; // found a repeat, stop looking
}
}

// if it is new, save it. if not, pick again.
if (!duplicate) {
a[filled] = candidate;
filled++;
}
}
return a;
}

public long getComparisons() {
return comparisons;
}

public static void main(String[] args) {
// make 25 numbers and print them
int n = 25;
Milestone_one gen = new Milestone_one(n);
int[] data = gen.generate();

System.out.println("n = " + n + ", values drawn from 1.." + (4 * n));
for (int i = 0; i < data.length; i++) {
System.out.print(data[i] + (i < data.length - 1 ? " " : "\n"));
}
System.out.println("Duplicate-check comparisons: " + gen.getComparisons());

// comparisons divided by n squared should stay about the same
System.out.println("\n n comparisons comparisons / n^2");
for (int m = 1000; m <= 16000; m *= 2) {
Milestone_one g = new Milestone_one(m);
g.generate();
double ratio = (double) g.getComparisons() / ((double) m * m);
System.out.printf("%6d %14d %14.4f%n", m, g.getComparisons(), ratio);
}
}
}
```

Output for n=25

```
n = 25, values drawn from 1..100
87 67 57 53 45 63 79 93 97 31 48 20 71 77 92 26 49 88 44 47 72 90 24 96 4
Duplicate-check comparisons: 330

n comparisons comparisons / n^2
1000 543283 0.5433
2000 2196995 0.5492
4000 8868653 0.5543
8000 35195926 0.5499
16000 141602036 0.5531
```

3.

Analysis

1. Basic operation: comparison A[j] = c , c(n) is the total number of times it runs
2. Cost of each step: I ≤ Ci< (4/3)i, for i = 0, ..., n−1
3. Add all the steps: Σ i = 0 , n-1 i = n(n−1)/2, so n(n−1)/2 ≤ E[C(n)] < 2n(n−1)/3
4. Bound it: for n ≥ 2, n(n−1)/2 ≥ n^2 /4 and 2n(n−1)/3 ≤ (2/3)n^2, so (1/4)n^2 ≤ E[C(n)] ≤ (2/3)n^2
5. Θ def: c1n^2 ≤ T(n) ≤ c2n^2, with c_1 = 1/4, c2 = 2/3, n0 = 2
6. Other work: random calls O(n) and filling the array O(n)
7. Result's: T(n) = Θ(n^2) + O(n) = Θ(n^2)